SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

SSAFY

SW문제해결 응용

컴퓨팅 사고력

Python

학습시작

'Problem Solving 문제를 봤는데 어떻게 풀어야 하는지 전혀 감이 오지 않는다면?

1. [ ] 정확하게 확인하는 과정에 대한 연습이 필요합니다
2. 정확하게 확인하는 과정을 수많은 세월 동안 정리해 둔 것이 [ ] 기법입니다

- Problem Solving에 필요한 기초 논리와 수학을 학습할 수 있어요.
- 문제들을 풀며 고민하는 과정을 통해 논리적 사고력이 향상할 수 있게 됩니다.
- 머리속으로 생각한 것을 실제 코드로 옮기는 연습을 할 수 있습니다.

Objectives

학습목표

- 문제 해결 방법을 논리적으로 설명할 수 있다.
- 명제와 논리, 증명의 기본 개념을 설명할 수 있다.
- 집합과 조합의 개념을 바탕으로 문제를 해결할 수 있다.
- 재귀 호출의 원리와 동작을 설명할 수 있다.
- 조합론을 적용해 문제를 해결할 수 있다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

컴퓨팅 사고력

- 프로그래밍과 논리/수학
- 논리와 증명
- 수와 표현
- 집합과 조합론
- 기초 수식
- 재귀
- 동적 계획법

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

컴퓨팅 사고력

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

프로그래밍과 논리/수학

논리의 정의

논리 (Hard Logic)

생각이나 주장의 타당성과 옳고 그름을 판별하는 규칙이나 원리를 의미합니다.

경험적, 직관적, 확률적 판단을 의미하는 Soft Logic에 반대되는 의미로 Hard Logic이라고 표현합니다.

맞는 이유와 틀린 이유를 정확히 밝히는 생각의 규칙입니다.

프로그래밍의 어려운 점

- 프로그래밍 언어 문법과 라이브러리 사용
  - 처음 보는 사람은 알 수 없습니다.
  - 훈련에 비례하여 실력이 늘게 됩니다.
- 논리
  - 직관적인 접근을 논리로 착각하기 쉽습니다.

카드 문제

- 직관으로 접근하면 어려운 문제
- 사실 : **모든 카드의 한쪽에는 알파벳이, 다른 쪽에는 숫자가 써 있음**
- 주장 : **한쪽이 D이면 반대쪽은 3**

- 주장이 사실인지 확인하기 위해 다음 카드들 중 반드시 뒤집어 보아야 하는 것은 몇 개이고 어느 것인가?

<그림1_SW문제해결 응용_컴퓨팅 사고력_카드 문제>

카드 문제

<그림1_SW문제해결 응용_컴퓨팅 사고력_카드 문제>

- **답 : [D]와 [7]**
- [D]를 뒤집어 보아야 한다는 것은 누구나 알아 냄
- [3]을 뒤집어 보아야 한다고 말하는 경우가 많이 있음
  - 그러나 [3] 뒤에 [D]가 있든 없든 주장이 사실인지 여부에 영향이 없습니다.
- [7]을 뒤집어 볼 필요가 없다고 말하는 경우도 많음
  - 그러나 [7] 뒤에 [D]가 있으면 주장이 성립하지 않게 됩니다.

명제에 대해 배우고 나면 다시 생각해 보세요

맥주집 문제

- 규칙 : 20세 이하인 사람은 맥주를 마실 수 없음
- 나이 혹은 마시고 있는 것을 표시한 다음 4명 중 확인이 필요한 사람은 몇 명이고 누구인가?
  - 답 : [17세]와 [맥주]

<그림2_SW문제해결 응용_컴퓨팅 사고력_맥주집 문제>

카드 문제와 맥주집 문제의 비교

- 맥주집 문제가 훨씬 풀기 쉬움
  - 사실, 두 문제는 완전히 같은 문제입니다. 즉, 논리적 구성이 완전히 동일합니다.
- 왜 맥주집 문제가 풀기 쉬운가?
  - **논리 구조를 정확히 이해하고 맥주집 문제를 푸는 사람은 카드 문제를 똑같이 풀 수 있습니다.**
  - 맥주집 문제를 풀 때 논리를 사용한 것이 아닙니다.

Soft Logic

- 맥주집 문제를 풀 때 직관을 사용한 것과 같은 경우
- 직관은 논리적인 느낌을 주는 것
- 직관의 장점은 익숙한 상황에서 빠름
- 직관의 단점은 정확하지 않음 (가끔은 익숙한 상황에서도 틀림)
- 또 다른 단점은 강한 착각을 일으킨다는 것

Hard Logic vs Soft Logic

- 일상 생활
  - Soft Logic이 빠르기 때문에 유용합니다.
  - 논리적으로 부정확한 표현을 사용하지만, 어떤 의미인지 모든 사람이 이미 알고 있다는 가정이 존재합니다.

- 프로그래밍
  - 프로그래밍 언어의 표현들이 모두 논리학에서 나온 것입니다.
  - 사용되는 수많은 알고리즘들을 이해하기 위해서는 Hard Logic이 필요합니다.

Hard Logic vs Soft Logic

- 문제에 대해 오해하는 경우

- Soft Logic으로 알고리즘을 이해하려고 하는 경우입니다.
- 알고리즘 설명을 보고 또 봐도 이해가 안되는 것은 증명을 안 봤기 때문입니다.
- 증명을 봐도 이해가 안 되는 것은 직관으로 이해하려고 하기 때문입니다.
- 가끔 직관적으로 이해되는 알고리즘이 있지만, 조금만 어려워지면 직관으로 완전한 이해를 얻는 것은 사실상 불가능합니다.

명제

- 참이나 거짓을 알 수 있는 식이나 문장
- p, q, r, …로 표현
- 예) 서울은 대한민국의 수도다.

  1+1=3

- 진릿값

 - 참이나 거짓을 표현
 - T, F 또는 1, 0
 - 예) p가 거짓(F) , q가 참(T)

명제의 연산(결합)

- 부정 (NOT)
  - p가 명제일 때, 명제의 진릿값이 반대가 됩니다.
  - ~p 또는 ¬p로 표기하고, not p 또는 p의 부정으로 읽습니다.

p	~p
T	F
F	T

<표1_SW문제해결 응용_컴퓨팅 사고력_명제의 부정>

- 논리곱 (AND)
  - p, q가 명제일 때, p, q 모두 참일 때만 참이 되는 명제입니다.
  - p ^ q로 표기하고, p and q 또는 p 그리고 q로 읽습니다.

p	q	p ^ q
T	T	T
T	F	F
F	T	F
F	F	F

<표2_SW문제해결 응용_ 컴퓨팅 사고력 _명제의 논리곱>

명제의 연산(결합)

- 논리합 (OR)

- p, q가 명제일 때, p, q 모두 거짓일 때만 거짓이 되는 명제입니다.
- p ∨ q로 표기하고 p or q나 p 또는 q로 읽습니다.

p	q	p ∨ q
T	T	T
T	F	T
F	T	T
F	F	F

<표3_SW문제해결 응용_컴퓨팅 사고력 _명제의 논리합>

명제의 연산(결합)

- 배타적 논리합 (XOR)

- p, q가 명제일 때, p, q 중 하나만 참일 때 참이 되는 명제입니다.
- p ⊕ q로 표기하고, p xor q 또는 p exclusive OR q로 읽습니다.

p	q	p ⊕ q
T	T	F
T	F	T
F	T	T
F	F	F

<표4_SW문제해결 응용_컴퓨팅 사고력_명제의 배타적 논리합>

조건명제

- p, q가 명제일 때, 명제 p가 조건(원인), q가 결론(결과)으로 제시되는 명제
- p → q (p이면 q이다)

p	q	p → q
T	T	T
T	F	F
F	T	T
F	F	T

<표5_SW문제해결 응용_컴퓨팅 사고력_조건명제>

쌍방조건명제

- p, q가 명제일 때, 명제 p와 q가 모두 조건이면서 결론인 명제
- p ↔ q (p면 q고, q면 p다)

p	q	p ↔ q
T	T	T
T	F	F
F	T	F
F	F	T

<표6_SW문제해결 응용_컴퓨팅 사고력_쌍방조건명제>

조건명제의 역, 이, 대우

- 주어진 명제 : p → q
- 역 : q → p
- 이 : ~p → ~q
- 대우 : ~q → ~p

p	q	p → q	q → p	~p → ~q	~q → ~p
T	T	T	T	T	T
T	F	F	T	T	F
F	T	T	F	F	T
F	F	T	T	T	T

<표7_SW문제해결 응용_컴퓨팅 사고력_명제의 역, 이, 대우>

문제 1 : 다음을 명제식 형태로 쓰고 참인지 거짓인지 판단하시오

① 만약 0이 홀수라면, 미국에서 2080년 월드컵이 열린다.

- p : 0은 홀수이다. (거짓)
- q : 미국에서 2080년 월드컵이 열린다. (알 수 없음)
- 명제식 : $p \rightarrow q$, p 명제가 거짓이므로, q명제의 참/여부에 상관없이 해당 명제식은 참이다.

② 만약 198938279382748390| Prime Number라면, 2는 짝수이다.

- p : 19893827938274839은 Prime Number이다. (알 수 없음)
- q : 2는 짝수이다. (참)
- 대우 명제는 $\sim q \rightarrow \sim p$ 인데, ~q 는 '2가 홀수이다' 가 되어 거짓인 명제가 된다. 따라서 $\sim q$ 명제가 거짓이므로, $\sim p$ 명제의 참/여부에 상관없이 해당 명제식은 참이 된다. 대우 명제식이 참이므로, 본 명제식 또한 참이다.

문제 2: p와 q가 명제이고 p → q가 거짓이라고 하자. 다음 명제식의 참 거짓은 어떻게 되는가?

① ~p → q

② p ∨ q

③ q → p

$p \rightarrow q$가 거짓이기 위해선 p 참, q 거짓인 경우이다.

① ~p → q : ~p는 거짓, q도 거짓이므로 조건 명제는 참
② p ∨ q : p가 참이므로 참
③ q → p : q는 거짓, p는 참이므로 참

문제 3: 다음 명제들의 역, 이, 대우를 쓰시오

① 만약 0이 홀수라면, 미국에서 2080년 월드컵이 열린다.

- 역 : 만약 미국에서 2080년 월드컵이 열린다면, 0이 홀수이다.
- 이 : 만약 0이 짝수라면, 미국에서 2080년 월드컵이 열리지 않는다.
- 대우 : 만약 미국에서 2080년 월드컵이 열리지 않는다면, 0은 짝수이다.

① 만약 198938279382748390| Prime Number라면, 2는 짝수이다.

- 역 : 만약 2가 짝수이면 198938279382748390| Prime Number 이다.
- 이 : 만약 198938279382748390| Prime Number가 아니라면 2는 홀수이다.
- 대우 : 만약 2가 홀수이면 198938279382748390| Prime Number가 아니다.

문제 4: 다음 명제식의 진리표를 만드시오

① $p \wedge (q \rightarrow \sim p)$

p	q	~p	$(q \rightarrow \sim p)$	$p \wedge (q \rightarrow \sim p)$

<그림3_SW문제해결 응용_컴퓨팅 사고력_문제4 진리표1>

② $(p \wedge \sim q) \rightarrow r$

p	q	r	~q	$(p \wedge \sim q)$	$(p \wedge \sim q) \rightarrow r$

<그림4_SW문제해결 응용_컴퓨팅 사고력_문제4 진리표2>

문제 4: 다음 명제식의 진리표를 만드시오 (풀이 1)

① $p \wedge (q \rightarrow \sim p)$

p	q	~p	(q→~p)	p∧(q→~p)
T	T	F	F	F
T	F	F	T	T
F	T	T	T	F
F	F	T	T	F

<표8_SW문제해결 응용_컴퓨팅 사고력_문제4 진리표1>

문제 4: 다음 명제식의 진리표를 만드시오 (풀이 2)

② $(p \wedge \sim q) \rightarrow r$

p	q	r	~q	(p∧~q)	(p∧~q)→r
T	T	T	F	F	T
T	T	F	F	F	T
T	F	T	T	T	T
T	F	F	T	T	F
F	T	T	F	F	T
F	T	F	F	F	T
F	F	T	T	F	T
F	F	F	T	F	T

<표9_SW문제해결 응용_컴퓨팅 사고력_문제4 진리표2>

증명

- 직접 증명
  - p → q가 참임을 증명하기 위해 p를 참으로 가정했을 때 q도 참임을 증명하는 방법입니다.
  - Trivial Proof(자명한 증명)
  - Vacuous Proof(공허한 증명)

- 간접 증명
  - 증명해야 하는 명제를 변형하여 증명하는 방법입니다.
  - 대우에 의한 증명
  - 모순에 의한 증명

- 수학적 귀납법
  - 일정한 규칙을 나타내는 명제 P(n)이 성립함을 증명하는 방법입니다.

증명

- 직접 증명
  - p → q가 참임을 증명하기 위해 p를 참으로 가정했을 때 q도 참임을 증명하는 방법입니다.
  - Trivial Proof(자명한 증명)
  - Vacuous Proof(공허한 증명)

- 간접 증명
  - 증명해야 하는 명제를 변형하여 증명하는 방법입니다.
  - 대우에 의한 증명
  - 모순에 의한 증명

- 수학적 귀납법
  - 일정한 규칙을 나타내는 명제 P(n)이 성립함을 증명하는 방법입니다.

Trivial Proof: ∀x,P(x)→Q(x)를 증명하려는데, Q(x)가 항상 참인 경우

- 다음 명제를 증명하시오

① 실수 x에 대해, 만약 x<-1이면 $x^2 + \frac{1}{4} > 0$이다

: x^2+1/4>0, x^2>−1/4 이고, x는 실수이므로 Q(x)는 항상 참이다. 따라서 $\forall x, P(x) \rightarrow Q(x)$ 이다.

② n이 홀수이면 $4n^3 + 6n^2 + 12$는 짝수이다

: $4n^3$+$6n^2$+12 = 2($2n^3$+$3n^2$+6) 이므로 $4n^3 + 6n^2 + 12$ 는 짝수이다. 그러므로 Q(x)는 항상 참이다.

따라서 $\forall x, P(x) \rightarrow Q(x)$ 이다.

Vacuous Proof: ∀x,P(x)→Q(x)를 증명하려는데, P(x)가 항상 거짓인 경우

- 다음 명제를 증명하시오

① 실수 x에 대해, 만약 $2x^2-4x+4<0$이면 x>8이다

: $2x^2-4x+4 = 2(x^2-2x)+4 = 2(x-1)^2+2 \geq 0$ 이다. 따라서 $2x^2-4x+4<0$은 거짓이다.

그렇기 때문에 $P(x)$는 거짓이므로 해당 명제 $\forall x, P(x) \rightarrow Q(x)$는 참이다.

② $4n^3+6n^2+11$ 는 짝수이면 n이 홀수이다

: $4n^3+6n^2+11=2(2n^3+3n^2+5)+1$ 이므로 $4n^3+6n^2+11$은 홀수이다. 그러므로 $P(x)$는 거짓이므로 해당 명제 $\forall x, P(x) \rightarrow Q(x)$는 참이다.

수학적 귀납법과 증명의 수준

- 0보다 크거나 같은 정수 범위에 대해 P(n)이 성립함을 증명

 - 수학적 귀납법의 기본형: P(1)이 참이고, P(n)→P(n+1)이 참이면 P(n)은 모든 자연수 n에 대해서 참이다.

 - 수학적 귀납법의 강한 형태: P(1)이 참이고, P(1)∧P(2)∧⋯∧P(n)→P(n+1)이 참이면 P(n)은 모든 자연수 n에 대해서 참이다.

- 다음 함수가 1부터 x까지의 합을 계산함을 증명한다면

```
def sum(x) :
    if (x <= 0):
        return 0
    return x + sum(x-1)
```

수학적 귀납법과 증명의 수준

• 상세한 증명을 하려면 단순히 “답이 맞는 것이 당연하다”라고 말하는 것으로는 충분하지 않음

- 증명이 가능한 명제를 만들어야 함.

- 이 경우 증명이 가능한 명제는 다음과 같음: “sum(x)가 리턴하는 값은 1+2+…+x의 값과 항상 같다”

- 이제 수학적 귀납법을 적용할 수 있음.

- P(1)이 참이다: “sum(1)이 리턴하는 값은 1이다”를 증명하면 됨. 실제 코드에 1을 대입하면 1을 리턴함을 알 수 있음.

```
sum(1)
    if (x <= 0) return 0
    return 1 + sum(0)
```

수학적 귀납법과 증명의 수준

- 임의의 값 k에 대해 P(k)가 참이라고 가정한다. : sum(k)가 1+2+…+k을 리턴
- k + 1에 대해 P(k+1)이 참이라고 증명한다.

```
sum(k+1) :
    if (k+1 <= 0) return 0
    return k+1 + sum(k)
```

- sum(k)는 1+2+…+k라고 가정했으므로, sum(k+1) =1+2+…+k+1을 리턴한다. P(k+1)이 참.
- 따라서 sum(x)는 1에서 x까지의 합을 리턴한다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

논리와 증명

문제 1: 다음 명제들이 항진명제라는 것을 진리표를 이용해서 보이시오

항진명제 :
명제의 진릿값과 무관하게
항상 참인 명제

<개념1_SW문제해결 응용_컴퓨팅사고력_항진명제>

① $\sim(\sim p \wedge q) \vee q$

p	q	~p	$(\sim p \wedge q)$	$\sim(\sim p \wedge q) \vee q$
T	T	F	F	T
T	F	F	F	T
F	T	T	T	T
F	F	T	F	T

<표10_SW문제해결 응용_컴퓨팅사고력_논리와 증명 문제1-1>

문제 1: 다음 명제들이 항진명제라는 것을 진리표를 이용해서 보이시오

② $(\sim p \vee q) \vee (p \wedge \sim q)$

p	q	$(\sim p \vee q)$	$(p \wedge \sim q)$	$(\sim p \vee q) \vee (p \wedge \sim q)$
T	T	T	F	T
T	F	F	T	T
F	T	T	F	T
F	F	T	F	T

<표11_SW문제해결 응용_컴퓨팅사고력_논리와 증명 문제1-2>

모순명제 :
명제의 진릿값과 무관하게
항상 거짓인 명제

<개념2_SW문제해결 응용_컴퓨팅사고력_항진명제>

문제 2: 다음 명제들이 모순명제라는 것을 진리표를 이용해서 보이시오

① $(\sim p \vee q) \wedge (p \wedge \sim q)$

p	q	$(\sim p \vee q)$	$(p \wedge \sim q)$	$(\sim p \vee q) \wedge (p \wedge \sim q)$
T	T	T	F	F
T	F	F	T	F
F	T	T	F	F
F	F	T	F	F

<표12_SW문제해결 응용_컴퓨팅사고력_논리와 증명 문제2-1>

문제 2: 다음 명제들이 모순명제라는 것을 진리표를 이용해서 보이시오

② $(p \wedge q) \wedge (p \wedge \sim q)$

p	q	$(p \wedge q)$	$(p \wedge \sim q)$	$(p \wedge q) \wedge (p \wedge \sim q)$
T	T	T	F	F
T	F	F	T	F
F	T	F	F	F
F	F	F	F	F

<표13_SW문제해결 응용_컴퓨팅사고력_논리와 증명 문제2-2>

문제 3: 다음 명제의 쌍 들에 대해서 두 명제가 동등한지를 진리표를 이용해 확인하시오

① $p \wedge (p \vee q)$와 p

p	q	$(p \vee q)$	$p \wedge (p \vee q)$
T	T	T	T
T	F	T	T
F	T	T	F
F	F	F	F

<표14_SW문제해결 응용_컴퓨팅사고력_논리와 증명 문제3-1>

문제 3: 다음 명제의 쌍 들에 대해서 두 명제가 동등한지를 진리표를 이용해 확인하시오

② $\sim p \vee \sim q$와 $\sim(p \vee q)$

p	q	$\sim p$	$\sim q$	$\sim p \vee \sim q$	$(p \vee q)$	$\sim(p \vee q)$
T	T	F	F	F	T	F
T	F	F	T	T	T	F
F	T	T	F	T	T	F
F	F	T	T	T	F	T

<표15_SW문제해결 응용_컴퓨팅사고력_논리와 증명 문제3-2>

문제 4: 명제식의 변형을 통하여 다음 명제를 간소화 하시오

① $(p \wedge \sim q) \vee (p \wedge q)$

$(p \wedge \sim q) \vee (p \wedge q) = p \wedge (\sim q \vee q) = p \wedge U = p$ (U는 항진명제)

② $(p \vee \sim q) \wedge (\sim p \vee \sim q)$

$(p \vee \sim q) \wedge (\sim p \vee \sim q) = (\sim q \vee p) \wedge (\sim q \vee \sim p) = \sim q \vee (p \wedge \sim p) = \sim q \vee \varnothing = \sim q$ ($\varnothing$는 모순 명제)

문제 5: 다음 명제들이 참인지 확인하시오 (단, R은 실수의 집합, Z는 정수의 집합)

① $\forall x \in R,\ x^2 \geq x$

x=1/2일 때, $x^2 \geq x$ 을 만족하지 않는다. (반례) 따라서 ① 명제는 거짓이다.

② $\forall x \in Z,\ x^2 \geq x$

해당 명제를 만족하지 않는 x의 범위는 $0 < x < 1$ 인데, 해당 범위엔 정수 값이 존재하지 않는다. 다시 말해 모든 정수 x에 대해 부등식 $x^2 \geq x$ 을 만족한다고 할 수 있다. 따라서 ② 명제는 참이다.

문제 5: 다음 명제들이 참인지 확인하시오 (단, R은 실수의 집합, Z는 정수의 집합)

③ $\exists x \in R,\ x^2 < x$

x=1/2일 때, $x^2 < x$ 을 만족한다. (어떤 x가 존재) 따라서 ③ 명제는 참이다.

④ $\exists x \in Z,\ x^2 < x$

$x^2 < x$, $x^2 - x < 0$, $x(x-1) < 0$ 이므로, 위 부등식의 해는 $0 < x < 1$가 된다. 이때, 해당 부등식 조건을 만족하는 어떤 정수도 존재하지 않는다. 따라서 ④ 명제는 거짓이다.

문제 6: (직접 증명) n이 짝수이면 3n+5는 홀수임을 증명하라

- 힌트: n=2k로 두고 3n+5가 2(어떤 정수)+1 형태로 표현될 수 있는가

Proof)

n = 2k 일 때(k는 정수), 3n + 5 = 3*(2k) + 5 = 6k + 5 = 6k + 4 + 1 = 2*(3k + 2) + 1

그러므로 3n + 5 는 홀수이다.

문제 7: n이 홀수이면 n^2+n은 짝수임을 증명하라

Proof)

n = 2k + 1 일 때 (k는 정수), $n^2+n=(2k+1)^2+(2k+1)=4k^2+4k+1+2+1= 4k^2+4k+4=2(2k^2+2k+2)$

그러므로 n^2+n은 짝수이다.

문제 8: m이 짝수이고 n이 홀수이면 $2m+3n$은 홀수임을 증명하라

Proof)

m = 2k, n = 2l+1 일 때(k, l은 정수),

2m + 3n = 2*(2k) + 3*(2l+1) = 4k + 6l + 3 = 2(2k + 3l + 1) + 1

그러므로 2m+3n은 홀수이다.

문제 9: (대우를 증명) 자연수 n에 대해, n^2+5가 홀수이면 n은 짝수임을 증명하라

Proof)

대우명제는, ‘n이 홀수이면 n^2+5은 짝수이다.’와 같다.

$n = 2k + 1$ 일 때, $n^2+5= (2k+1)^2+5= 4k^2+4k+6=2(2k^2+2k+3)$

그러므로 n이 홀수이면 n^2+5는 짝수이다.

대우 명제가 참이므로, 본 명제 또한 참이다.

문제 10: n^2이 짝수이면 n은 짝수임을 증명하라

Proof)

주어진 명제의 대우명제는, 'n이 홀수이면 n^2이 홀수이다.'와 같다.

n = 2k + 1일 때, $n^2 = (2k+1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1$

그러므로 n이 홀수이면 n^2이 홀수이다.

대우 명제가 참이므로, 본 명제 또한 참이다.

문제 11: (경우를 나누어 증명) 자연수 n에 대해 n^2+5n+3은 항상 홀수임을 증명하라.

Proof)

n이 짝수인 경우

n = 2k (k>=1), $n^2+5n+3= (2k)^2+5(2k)+3=4k^2+10k+3=2(2k^2+5k+1)+1$

그러므로, n이 짝수인 경우 n^2+5n+3은 항상 홀수이다.

n이 홀수인 경우

n = 2k+1 (k>=0), $n^2+5n+3= (2k+1)^2+5(2k+1)+3=4k^2+14k+9=2(2k^2+7k+4)+1$

그러므로, n이 홀수인 경우 n^2+5n+3은 항상 홀수이다.

n이 짝수인 경우와 홀수인 경우 모두 n^2+5n+3은 항상 홀수이므로, 자연수 n에 대해 n^2+5n+3은 항상 홀수이다.

문제 12: n^2이 3의 배수이면 n은 3의 배수임을 증명하라 (과제)

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

수와 표현

약간의 설명

- 컴퓨터는 0/1을 표현할 수 있는 비트들을 모아 수를 표현
- k개의 비트를 사용하면 0부터 2^K-1까지 표현 가능
- 사실, 꼭 저 범위인 것은 아님. 약속하는 방식에 따라 다르지만, 최대 2^k 가지의 값을 표현하는 것이 가능
- 10진수로 k자리를 쓰면 0부터 10^k-1까지 표현이 가능한 것과 완전히 동일한 과정
- 어떤 값 n을 표현하기 위해서는 몇 개의 비트가 필요할까?

 - $2^k-1 \geq n$이 성립해야 합니다. -> 즉, $2^k \geq n+1$

 - 같은 의미로, $k \geq \log(n+1)$ -> 약 $\log n$ 비트가 필요합니다.

- $x=\log n$과 $2^x=n$은 같은 말

log n 이란

- 2의 몇 승이 n이 되는가의 답
- n을 표현하는 데 몇 비트가 필요한가의 답
- 1로 시작해서 계속 두 배를 할 때 몇 번 하면 n이 되는가의 답
- n을 2로 계속 나눌 때 몇 번 나누면 거의 1이 되는가에 대한 답
- x=log n일 때 x와 n을 비교하면 x가 더 작고, n이 커질수록 엄청나게 달라짐

  - 100자리 n은 읽을 수도 없을 정도로 큰 값이지만 x는 작은 값입니다.

- 컴퓨터 분야에서 로그의 밑은 항상 2

log n 이란

- 32비트 컴퓨터의 주소 공간은 2^{32}= 약 40억개
- $n+\left(\frac{n}{2}+\frac{n}{2}\right)+\left(\frac{n}{4}+\frac{n}{4}+\frac{n}{4}+\frac{n}{4}\right)+\left(\frac{n}{8}+\frac{n}{8}+\cdots\right)+\cdots(1+1+\cdots)\cong n\log n$
- n+n/2+n/4+…+1 ≅ 2n
- 위 두 식에서 항의 개수는 log n개.

 - 분모를 2^k 로 나타내면, 1 = $n/2^k$, 2^k = n 이므로 k = log n 입니다.

문제 1: 2진수 표현에서 log n 비트로 표현할 수 있는 숫자 범위는?

- n비트로 표현할 수 있는 숫자는 2^n가지
- $\log_2 n$ 비트로 표현할 수 있는 숫자는 n가지

문제 2: 스무고개가 이상적으로 진행된다고 할 때, 맞출 수 있는 답의 종류는 몇 가지인가?

- 2^{20}= 1048576 가지

문제 3: n이 충분히 큰 값일 때 다음 중 어느 값이 더 큰가? 각 쌍에 대해 비교하고 그 이유를 작성하시오

① $2n$ (<) n^2

: 2<n 일 때 2n< n^2 이므로, 2보다 큰 모든 n에 대해 부등호가 항상 성립한다.

② $2^{\frac{n}{2}}$ (<) $\sqrt{3^n}$

: n>0 일 때 $3^{n/2}>2^{n/2}$ 이 항상 성립하므로, 0보다 큰 모든 n에 대해 부등호가 항상 성립한다.

문제 3: n이 충분히 큰 값일 때 다음 중 어느 값이 더 큰가? 각 쌍에 대해 비교하고 그 이유를 작성하시오

③ $2^{n \log n}$ (>) $n!$

: n > 1일 때, $n^n > n!$ 이 항상 성립한다. 따라서 1보다 큰 모든 n에 대해 부등호가 항상 성립한다.

④ $\log 2^{2n}$ (<) $n\sqrt{n}$

: n > 4 일 때 $2n < n^{3/2}$ 이 항상 성립한다. 따라서 4보다 큰 모든 n에 대해 부등호가 항상 성립한다.

문제 4: x=log_ayz일 때 x를 2를 밑으로 하는 로그들로 표현하시오. 단, 로그 함수의 인자는 모두 문자 하나여야 한다.

- 로그의 성질에 대해 찾아볼 것

$$x = \frac{\log_2 y + \log_2 z}{\log_2 a}$$

문제 5: 다음 함수들의 역함수를 구하시오

① $f(x)=\log(x-3)-5$

② $f(x)=3\log(x+3)+1$

③ $f(x)=2\times3^{x}-1$

$$f(x)=\log(x-3)-5$$
$$2^{f(x)+5}=x-3$$
$$\mathrm{x}=2^{f(x)+5}+3$$
$$f^{-1}(x)=2^{x+5}+3$$

$$f(x)=3\log(x+3)+1$$
$$2^{\frac{f(x)-1}{3}}=x+3$$
$$\mathrm{x}=2^{\frac{f(x)-1}{3}}-3$$
$$f^{-1}(x)=2^{\frac{x-1}{3}}-3$$

$$f(x)=2\times3^{x}-1$$
$$\mathrm{x}=\log_3(f(x)+1)-\log_3 2$$
$$f^{-1}(x)=\log_3(x+1)-\log_3 2$$

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

집합과 조합론

집합과 조합론에 대한 약간의 설명

- 두 집합 A와 B에 대해 A가 B의 부분집합임을 증명한다는 것은 A의 임의의 원소가 B에 포함됨을 보이는 것과 같다.
- 예를 들어 모든 4의 배수는 2의 배수라는 것을 증명하려면, 4k=2(2k)임을 보이면 되는 것이다.
- 두 집합 A와 B가 같다는 것을 증명하기 위해서는 A가 B의 부분집합이고 B가 A의 부분집합임을 증명하면 된다.

다음 두 집합이 같다는 것을 상세히 증명

- A={x|x=2k+1,k는 자연수}, B={x|x=4k+1 혹은 x=4k+3,k는 자연수}
- **A가 B의 부분집합이다:**
  - A에 포함되는 임의의 원소 x를 가정.
  - x=2k+1임.
  - k가 짝수(=2t)인 경우와 홀수(=2t+1)인 경우로 나눔.
  - 짝수인 경우 x=2k+1=2(2t)+1=4t+1로서, x는 B에 포함됨.
  - 홀수인 경우 x=2k+1=2(2t+1)+1=4t+3로서, x는 B에 포함됨.
  - 모든 가능한 경우에 x는 B에 포함됨.
- **B가 A의 부분집합이다:**
  - B에 포함되는 임의의 원소 x를 가정.
  - x=4k+1인 경우, x=4k+1=2(2k)+1로서 x는 A에 포함됨. x=4k+3인 경우, x=4k+3=2(2k+1)+1로서 x는 A에 포함됨.
  - 모든 가능한 경우에 x는 B에 포함됨.
- **위 두 가지 증명에서 집합 A와 B는 같다.**

조합론

- 조합론은 경우의 수를 따지는 문제들을 말함
- 조합은 개수는 C를 이용하여 ${}_5C_2$ 처럼 표현하기도 하지만 $\binom{5}{2} = 10$과 같은 괄호 표현을 더 많이 씀

문제 1 : $\binom{n}{k} + \binom{n}{k-1} = \binom{n+1}{k}$임을 증명하라

Proof)

$$\binom{n}{k} + \binom{n}{k-1} = \frac{n!}{(n-k)!k!} + \frac{n!}{(n-(k-1))!(k-1)!}$$

$$= n!\left(\frac{1}{(n-k)!\,k!} + \frac{1}{(n-(k-1))!\,(k-1)!}\right)$$

$$= n!\left(\frac{n-(k-1)}{(n-(k-1))!\,k!} + \frac{k}{(n-(k-1))!\,k!}\right)$$

$$= n!\left(\frac{n+1}{(n-(k-1))!\,k!}\right)$$

$$= \frac{(n+1)!}{(n+1-k)!\,k!}$$

$$= \binom{n+1}{k}$$

문제 2 : 수학적 귀납법으로 $(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^{n-k}y^k$ 임을 증명하라 (1)

Proof)

$n = 1$ 일 때,

$$(\text{좌변}) = (x+y)^1 = \sum_{k=0}^{1} \binom{1}{k} x^{1-k}y^k = (\text{우변})$$

이므로 성립한다.

$n = t\ (t \geq 1)$ 일 때, 등식이 성립한다고 가정하면,.

$$(x+y)^t = \sum_{k=0}^{t} \binom{t}{k} x^{t-k}y^k$$

$n = t+1$ 일 때,

$$(x+y)^{t+1} = (x+y)^t(x+y) = \left(\sum_{k=0}^{t} \binom{t}{k} x^{t-k}y^k\right)(x+y)$$

$$= \sum_{k=0}^{t} \binom{t}{k} x^{t-k+1}y^k + \sum_{k=0}^{t} \binom{t}{k} x^{t-k}y^{k+1}$$

문제 2 : 수학적 귀납법으로 $(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^{n-k} y^k$ 임을 증명하라 (2)

$$= \left(\binom{t}{0} x^{t+1} + \sum_{k=1}^{t} \binom{t}{k} x^{t-k+1} y^k \right) + \left(\binom{t}{t} y^{t+1} + \sum_{k=0}^{t-1} \binom{t}{k} x^{t-k} y^{k+1} \right)$$

여기서

$$\sum_{k=0}^{t-1} \binom{t}{k} x^{t-k} y^{k+1} = \sum_{k=1}^{t} \binom{t}{k-1} x^{t-(k-1)} y^k \text{ 이고,}$$

이항 계수 성질에 의하여,

$$\binom{t}{k} + \binom{t}{k-1} = \binom{t+1}{k}, \quad \binom{t}{0} = \binom{t+1}{0}, \quad \binom{t}{t} = \binom{t+1}{t+1}$$

이므로

문제 2 : 수학적 귀납법으로 $(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^{n-k}y^k$ 임을 증명하라 (3)

$$(x+y)^{t+1} = \binom{t+1}{0}x^{t+1} + \binom{t+1}{k+1}y^{t+1} + \sum_{k=1}^{t}\binom{t}{k}x^{t-k+1}y^k + \sum_{k=1}^{t}\binom{t}{k-1}x^{t-k-1}y^k$$

$$= \binom{t+1}{0}x^{t+1} + \binom{t+1}{k+1}y^{t+1} + \sum_{k=1}^{t}\left[\binom{t}{k} + \binom{t}{k-1}\right]x^{t-k+1}y^k$$

$$= \binom{t+1}{0}x^{t+1} + \binom{t+1}{k+1}y^{t+1} + \sum_{k=1}^{t}\binom{t+1}{k}x^{t-k+1}y^k$$

$$= \sum_{k=0}^{t+1}\binom{t+1}{k}x^{t-k+1}y^k$$

즉, $n = t+1$ 일 때도 주어진 등식이 성립함을 알 수 있다.
따라서 주어진 등식은 모든 자연수 n에 대하여 성립한다.

문제 3: 위의 결과를 이용해서 n개의 원소를 가진 집합의 가능한 부분집합의 종류는 2^n개임을 증명하라

Proof)

부분 집합의 원소의 개수를 총 n개라고 했을 때,

원소의 개수가 0개인 부분집합의 수 : $\binom{n}{0}$

원소의 개수가 1개인 부분집합의 수 : $\binom{n}{1}$

원소의 개수가 2개인 부분집합의 수 : $\binom{n}{2}$

…

원소의 개수가 n개인 부분집합의 수 : $\binom{n}{n}$

원소의 개수가 n개일 때, 총 부분집합의 수 S는 $\binom{n}{0} + \binom{n}{1} + \binom{n}{2} + \cdots + \binom{n}{n}$ 이다.

$$S = \binom{n}{0} + \binom{n}{1} + \binom{n}{2} + \cdots + \binom{n}{n} = \sum_{k=0}^{n} \binom{n}{k} 1^{n-k} 1^k = (1+1)^n = 2^n$$

이므로 n개의 원소를 가진 집합의 가능한 부분집합의 종류는 2^n개이다.

문제 4: 귀류법을 이용하여 $(A-B)\cap(B-A)=\varnothing$임을 증명하라 (1)

귀류법 : 간접증명의 한 방식

한 명제가 참인 것을 증명하려고 할 때에, 그 명제의 부정(否定)을 참이라고 가정(假定)하여 거기에서 나타나는 불합리성을 증명함으로써 원래의 명제가 참인 것을 보여 주는 증명법 또는 P^~Q를 증명하는 방법

Proof)
$p \in (A-B)$ 이고 $p \in (B-A)$ 인 원소 p가 존재한다고 가정하자. … ①

1. $p \in \mathrm{A}$, $p \notin \mathrm{B}$ 일 경우
$(B-A)$는 집합 B의 부분 집합이다. $p \notin \mathrm{B}$ 이므로 $p \notin (B-A)$ 이다. 이는 가정 ①의 모순이다.

문제 4: 귀류법을 이용하여 $(A-B)\cap(B-A)=\varnothing$임을 증명하라 (2)

2. $p \notin \mathrm{A}$, $p \in \mathrm{B}$ 일 경우
$(A-B)$는 집합 A의 부분 집합이다. $p \notin \mathrm{A}$ 이므로 $p \notin (A-B)$ 이다. 이는 가정 ①의 모순이다.

3. $p \notin \mathrm{A}$, $p \notin \mathrm{B}$ 일 경우
$(A-B)$와 $(B-A)$는 집합 A, B의 부분 집합이다. $p \notin \mathrm{A}$, $p \notin \mathrm{B}$ 이므로 $p \notin (A-B)$ 이고 $p \notin (B-A)$이다. 이는 가정 ①의 모순이다.

4. $p \in \mathrm{A}$, $p \in \mathrm{B}$ 일 경우
$p \in A \cap B$ 이므로, $p \notin (A-B)$이고 $p \notin (B-A)$ 이다. 이는 가정 ①의 모순이다.

따라서 $p \in (A-B)$ 이고 $p \in (B-A)$ 인 원소 p가 존재한다는 가정은 모순이므로, $p \in (A-B)$ 이고 $p \in (B-A)$ 인 원소 p는 존재하지 않는다.

그러므로 $(A-B)\cap(B-A)=\emptyset$ 이다.

문제 5: 다음이 사실임을 증명하라

$$(A \cup B) \cap (A \cap B)^c = (A - B) \cup (B - A)$$

Proof)

$$(A \cup B) \cap (A \cap B)^c = \{\, p : (p \in A \vee p \in B) \wedge \neg(p \in A \wedge p \in B) \,\}$$
$$(A - B) \cup (B - A) = \{\, p : (p \in A \wedge p \notin B) \vee (p \notin A \vee p \in B) \,\}$$
$$(p \in A \vee p \in B) \wedge \neg(p \in A \wedge p \in B) \Leftrightarrow (p \in A \wedge p \notin B) \vee (p \notin A \vee p \in B)$$

를 보이면 된다.

$$\equiv p \in (A \cup B) \cap (A \cap B)^c$$
$$\equiv (p \in A \vee p \in B) \wedge \neg(p \in A \wedge p \in B)$$
$$\equiv (p \in A \vee p \in B) \wedge (p \notin A \vee p \notin B)$$
$$\equiv \Big((p \in A \wedge (p \notin A \vee p \notin B)) \vee (p \in B \wedge (p \notin A \vee p \notin B)) \Big)$$
$$\equiv ((p \in A \wedge p \notin B) \vee (p \notin A \wedge p \in B)$$
$$\equiv p \in (A - B) \cup (B - A)$$

임을 알 수 있다.
또는
1. $p \in A$, $p \notin B$ 일 경우
2. $p \notin A$, $p \in B$ 일 경우
3. $p \notin A$, $p \notin B$ 일 경우

로 가정하여, 증명할 수 있다. 이는 문제 4번과 유사하게 증명해 나가면 된다.

문제 6: 8×8 체스 판에 말 두개를 놓으려고 한다. 아무 곳에나 놓아도 되지만 한 칸에 두개가 들어가지는 못한다. 가능한 방법은 모두 몇가지인가?

8×8 체스 판 중에서 두 곳을 고르는 조합을 구하는 문제이므로,

$$\binom{64}{2} = 2{,}016$$

모두 $2{,}016$개가 존재한다.

문제 7: 비밀번호를 0부터 9까지의 숫자만 가지고 만든다고 하자. 4개 이상 6개 이하의 숫자를 쓸 수 있다고 할 때 가능한 비밀번호의 가지수는 얼마인가?

가능한 비밀번호는 숫자가 4개, 5개, 6개가 들어갈 경우이다.

숫자가 6개 사용될 경우 : $10 \times 9 \times 8 \times 7 \times 6 \times 5 = 151{,}200$
숫자가 5개 사용될 경우 : $10 \times 9 \times 8 \times 7 \times 6 = 30{,}240$
숫자가 4개 사용될 경우 : $10 \times 9 \times 8 \times 7 = 5{,}040$

따라서 총 가능한 비밀번호의 가지 수는 186,480개 이다.

문제 8: 52개의 카드를 이용해서 만들 수 있는 5개 카드의 조합은 몇가지인가?

52개의 카드에서 5개 카드를 선택하는 조합의 경우이므로,
총 $\binom{52}{5} = 2{,}598{,}960$ 가지가 있다.

문제 9: 52개의 카드를 이용해서 만들 수 있는 5개 카드 조합 중 같은 무늬의 카드가 정확히 3개인 경우는 몇가지인가?

무늬의 종류는 4가지이고, 이 중 하나를 선택하는 경우는 $\binom{4}{1}$ 가지이다. 그리고 같은 무늬일 때, 서로 다른 숫자를 뽑는 경우는 13개 중 3개를 뽑는 경우 $\binom{13}{3}$ 이다.

나머지 2개의 카드는 무늬가 달라야 하므로, 나머지 3개에서 2개를 뽑는 $\binom{3}{2}$ 가지이고, 수는 서로 상관없으므로 $\binom{13}{1} \times \binom{13}{1}$이다.

따라서 이들을 모두 곱하면,
$\left(\binom{4}{1} \times \binom{13}{3}\right) \times \left(\binom{3}{2} \times \binom{13}{1} \times \binom{13}{1}\right) = 580{,}008$ 가지가 된다.

문제 10: x+y+z=100의 자연수 해는 몇가지인가?

1열로 늘어선 100개의 공사이에 2개의 칸막이를 세우는 것과 같다.

즉, 공과 공 사이 공간 99개에 중 2개를 골라 칸막이를 세우는 것과 같다.

$$\binom{99}{2} = 4851$$

문제 11: 52개 카드에서 5개 카드 조합을 만들 때, 숫자가 같은 카드가 한 쌍도 없는 경우는 몇가지인가?

(1) 52C1 : 1장 고르고,
(2) 48C1 : (1)과 같은 숫자 4장 빼고 1장 고르고
(3) 44C1
(4) 40C1
(5) 36C1
(1)*(2)*(3)*(4)*(5)/ 5! : 고른 순서에 의해 구분되는 경우 제거
정답은 1,317,888 입니다.

다른 방법
13C5 서로 다른 숫자를 고르고
13C5 * 4^5 각 숫자가 가질 수 있는 무늬(4)만큼 경우의 수를 곱합

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

기초 수식

약간의 설명

- 알고리즘의 시간 복잡도를 표현할 수 있는 다양한 수식들이 존재함
- 풀이법을 익혀 두어야 알고리즘의 시간 복잡도를 계산할 수 있고, 알고리즘이 시간이 얼마나 걸릴 지 예측할 수 있음
- 재귀식을 O( ) notation 수준으로 풀어보는 연습이 필요

문제 1: $T(n)=T(n-1)+1, T(0) = 1$

$$
\begin{aligned}
T(n) &= T(n-1)+1 \\
&= T(n-2)+1+1 \\
&= T(n-k)+k \\
&= T(0)+n, n == k \\
&= 1+n
\end{aligned}
$$

$$O(T(n)) = \boldsymbol{O(n)}$$

문제 2: $T(n)=T(n-1)+n$, $T(0)=1$

$$
\begin{aligned}
T(n) &= T(n-1)+n \\
&= T(n-2)+(n-1)+n \\
&= T(n-k)+\big(n-(k-1)\big)+\ \ldots+n \\
&= T(0)+1+\ \ldots+n, n == k \\
&= 1+\frac{n(n+1)}{2}
\end{aligned}
$$

$$O(T(n)) = \boldsymbol{O(n^2)}$$

문제 3: $T(n)=T(n-1)+\log n,\ T(0)=1$

$$
\begin{aligned}
T(n) &= \mathrm{T}(0) + \log n + \log(n-1) + \log(n-2) + \cdots + \log 1 \\
&\leq \mathrm{T}(0) + \log n + \log n + \log n + \cdots + \log n \\
&\leq 1 + n\log n \\
\mathrm{O}(T(n)) &= \boldsymbol{O(n \log n)}
\end{aligned}
$$

문제 4: $T(n)=T(n/2)+1$, $T(1)=1$ (과제)

문제 5: $T(n)=T(n/2)+n$, $T(1)=1$

$T(n) = n + T\left(\frac{n}{2}\right)$, $n>1$

$= n + \left(\frac{n}{2} + T\left(\frac{n}{4}\right)\right)$

$= n + \frac{n}{2} + \frac{n}{2^2} + \cdots + \frac{n}{2^{k-1}} + T\left(\frac{n}{2^k}\right)$

$T\left(\frac{n}{2^k}\right) = T(1)$

$2^k = n \Rightarrow k = \log n$

= n(1 + 1/2 + ⋯ + 1/2^log(n − 1)) + 1 <- (1/2+1/4+1/8+⋯ 다 더해도 1)
= 2n + 1

혹은 등비급수로 표현시 2n-1

$O(T(n)) = \boldsymbol{O(n)}$

문제 6: $T(n)=2T(n/2)+n$, $T(1)=1$

$T(n) = n + 2T\left(\frac{n}{2}\right)$

$= n + 2\left(\frac{n}{2} + 2 \cdot T\left(\frac{n}{4}\right)\right)$

$= n + 2 \cdot \frac{n}{2} + 2^2 \cdot \frac{n}{2^2} + \cdots + 2^{k-1} \cdot \frac{n}{2^{k-1}} + 2^k T\left(\frac{n}{2^k}\right)$

T(1) => $\frac{n}{2^k} = 1$ => k = log n, $2^k = 2^{\log n} = n$

$= n \log_2 n$

$O(T(n)) = \boldsymbol{O(n \log n)}$

문제 7: $T(n)=3T(n/2)+n$, $T(1)=1$

$$T(n) = n + 3T\left(\frac{n}{2}\right)$$
$$= n + 3\left(\frac{n}{2} + 3 \cdot T\left(\frac{n}{4}\right)\right)$$
$$= n + 3 \cdot \frac{n}{2} + 3^2 \cdot \frac{n}{2^2} + \cdots + 3^{k-1} \cdot \frac{n}{2^{k-1}} + 3^k T(\frac{n}{2^k})$$

k = log n,

$$3^k = 2^k \left(\frac{3}{2}\right)^k = 2^{logn} \left(\frac{3}{2}\right)^k = n\left(\frac{3}{2}\right)^k$$

$$\frac{n(r^k - 1)}{r - 1} = n\frac{\left(\frac{3}{2}\right)^k - 1}{\frac{3}{2} - 1} = 2n\left(\frac{3}{2}\right)^k - 2\mathrm{n}$$

$$2n\left(\frac{3}{2}\right)^{logn} - 2\mathrm{n} = 2n \cdot n^{\log\frac{3}{2}} - 2\mathrm{n} = 2n^{\log 3 - \log 2} - 2n = 2n\frac{n^{\log 3}}{n^{\log 2}} - 2n = 2n^{\log 3} - 2n$$

$$O(T(n)) = \boldsymbol{O\left(n^{\log_2 3}\right)}$$

등비수열
$a, ar, ar^2, ar^3, \ldots$ 초항부터 n항까지의 합
$\frac{a(1-r^n)}{1-r}$ 편의상 $\frac{a(r^n-1)}{r-1}$

문제 8: $T(n)=T(n-1)+1/n$, $T(1)=1$

$$T(n) = T(n-1) + \frac{1}{n}$$

$$= T(n-2) + \frac{1}{n-1} + \frac{1}{n}$$

...

$$= T(1) + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \frac{1}{6} + \frac{1}{7} \cdots \frac{1}{n-2} + \frac{1}{n-1} + \frac{1}{n}$$

$$< T(1) + (\frac{1}{2} + \frac{1}{2}) + (\frac{1}{4} + \frac{1}{4} + \frac{1}{4} + \frac{1}{4}) \cdots (\frac{1}{n} + \cdots + \frac{1}{n} + \frac{1}{n})$$

$$= T(1) + 1 + 1 + \cdots + 1 = T(1) + \boldsymbol{O(\log n)}$$

마지막 ()안의 분모를 2^k으로 표시했을 때 , ()의 개수는 k개, $n=2^k$이므로 k=logn

재귀

약간의 설명

- 재귀란 자기 자신을 호출하는 함수, 그럼 끝날 수가 있는가?
- 함수는 입력이 있으며, 자기 자신의 입력과 동일한 입력으로 자기 자신을 호출하면 당연히 끝나지 않음
- 하지만, 다른 입력으로 호출하면 끝날 수 있음

```
def abc(x):                          # 이 함수는 안 끝남
    return abc(x)


def sum(x):                 # 이 함수는 끝남. 결과 값은?
    if (x <= 0):
        return 0
    return x + sum(x-1)
```

약간의 설명

- 함수란 어떤 문제를 해결하는 방법을 코딩한 것
- 함수가 어떤 문제의 단 한 케이스만을 해결하는 것이 아님
- 제대로 코딩 된 것이라면 해결하는 문제의 모든 케이스들을 해결해야 함
- 수학적 귀납법 증명 사용 가능

- (가) n이 0일 때 문제를 풀 수 있음
- (나) n-1에서 문제를 풀 수 있으면 n에서도 문제를 풀 수 있다
- 위 두 가지가 사실이면 모든 가능한 n에 대해 문제를 풀 수 있다는 것이 사실

이론

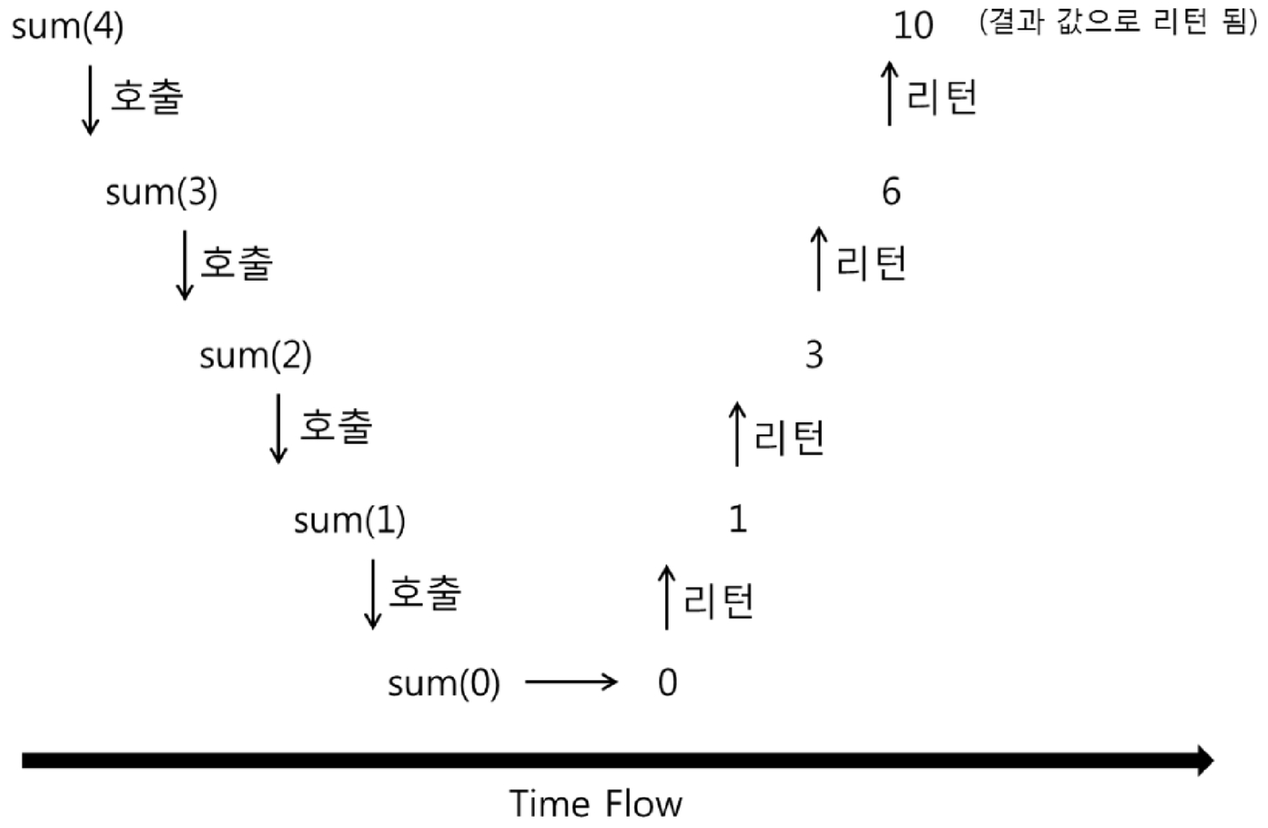
약간의 설명

- 위 박스의 함수 sum()을 보면 정확히 앞의 두 가지를 코딩한 것임
- 따라서, sum() 함수는 문제를 해결한다는 것을 증명할 수 있음
- 방금 보인 증명은 high-level 증명이며, 상세한 증명은 앞에서 본 적이 있음
- 어떤 미스터리한 이유로 문제가 해결되는 것은 아니고, 실제로 프로그램을 돌리면 필요한 계산이 다 일어남. 순차적인 코드에서 일어나는 계산과 완전히 동일. 다만 표현하는 방법이 달라진 것
- 다르게 생각하는 방법

- 어떤 문제를 해결하려다 부분 문제를 만났는데, 원래 해결하려던 입력 케이스와 동일한 문제에 속하지만 “크기가 더 작은” 입력 케이스를 해결하는 것이 그 부분 문제였다!

- 즉, 부분 문제가 동일한 문제인 경우!

sum()함수의 동작



<그림3_SW문제해결 응용_컴퓨팅 사고력_재귀함수의 예 >

```
def sum(x):
    if (x <= 0):
        return 0
    return x + sum(x-1)
```

재귀에 대한 연습

- 재귀 알고리즘을 **슈도코드**로 작성하고, **정확성 증명** 및 **시간 복잡도 계산**을 수행

문제 1: 피보나치 수열: F(n)=F(n-1)+F(n-2), F(1)=F(2)=1

- 슈도코드

```
fibo(n)
    if(n <= 2 )
        return 1
    return fibo(n-1) + fibo(n-2)
```

문제 1: 피보나치 수열: F(n)=F(n-1)+F(n-2), F(1)=F(2)=1

- 증명 및 시간 복잡도

Proof)
가정 : fibo(n)을 호출하면 F(n)이 return됨을 알 수 있다.
$n = 1$, n=2일 때,
$\text{fibo}(1) = 1$ 이고 fibo(2)=1인데, 이는 F(1) = 1, F(2) = 1이므로 위의 소스 코드는 성립한다.
$n = k$일 때,
fibo(1) … fibo(k-1)이 F(1) .. F(k-1)과 같다고 가정하자. 이때 F(k) = F(k-1)+ F(k-2) 이므로, fibo(k) = fibo(k-1)+fibo(k-2) = F(k)가 성립한다. 따라서 fibo(n)을 호출하면 F(n)이 return됨을 알 수 있다.

Time Complexity)

$$T(n) = T(n-1) + T(n-2) + 1$$
$$< 2\,T(n-1) + 1, T(0) = 1$$

$$O(T(n)) = O(2^n)$$

$$2T(n-1) + 1$$
$$= 2(2T(n-2) + 1) + 1$$
$$= 2^k T(n-k) + 2^{k-1} + 1$$
$$= 2^n\, T(0) + 2^{n-1} + 1\,, n == k$$

문제 2: Merge Sort

- 슈도코드

```
mergesort(array, buffer, start, end){
        if(start == end) return
        mid = (start+end)/2
        i0 = start
        i1 = mid+1
        mergesort(array, buffer, start, mid)
        mergesort(array, buffer, mid+1, end)

        for(i from start to end){
                if(i1 > end || i0 <= mid && array[i0] > array[i1])
                        buffer[i] = array[i0++]
                else
                        buffer[i] = array[i1++]
        }
        copy buffer to array
}
```

문제 2: Merge Sort

- 증명 및 시간 복잡도

Proof)
mergesort함수는 주어진 배열을 소팅한다.
k = start – end라고 할 때, k=1 및 k=2 일 때 Merge Sort가 배열을 소팅함을 알 수 있다.
k=1 , k=2, … ,k=n에서 Merge Sort가 배열을 소팅한다고 가정하면 k=n+1일 때 Merge Sort가 배열을 소팅함을 코드를 확인해서 증명할 수 있다. (Merge 알고리즘의 정확성을 사용함)
따라서 모든 범위 k에서 Merge Sort가 성립함을 알 수 있다.

$$
\begin{aligned}
T(n) &= n + 2T\left(\frac{n}{2}\right) \\
&= n + 2\left(\frac{n}{2} + 2 \cdot T\left(\frac{n}{4}\right)\right) \\
&= n + 2 \cdot \frac{n}{2} + 2^2 \cdot \frac{n}{2^2} + \cdots + 2^{\log_2 n} \cdot \frac{n}{2^{\log_2 n}} \\
&= \mathrm{n}\log_2 n
\end{aligned}
$$

$$
\mathrm{O}(T(n)) = \boldsymbol{O}(\mathbf{n}\log \boldsymbol{n})
$$

문제 3: 다음 소팅 알고리즘이 실제로 소팅에 항상 성공한다는 것을 증명하라

```
Stupid (A[0..n-1])
{
    if n=2 and A[0] > A[1]
        then swap A[0] and A[1]
    else
        m = ceiling(2n/3)
        Stupid(A[0..m-1])
        Stupid(A[n-m..n-1])
        Stupid(A[0..m-1])
}
```

Proof)
배열 A[0..n-1]을 [0,n-m-1], [n-m,m-1], [m,n-1]의 3 구간으로 나눌 수 있다. 각각의 구간을 A,B,C 구간이라고 할 때, 위의 소팅 방법은
.. ①[A B], ②[B C], ③[A B]의 순서로 소팅이 됨을 알 수 있다.

기저 조건에 대해 증명하고, 원소의 개수가 2부터 n-1까지의 소팅이 성립한다고 가정할 때 개수가 n일때의 소팅이 성립함을 보이는 귀납적 방법으로 위의 stupid소팅법을 증명할 수 있다.

문제 3: 다음 소팅 알고리즘이 실제로 소팅에 항상 성공한다는 것을 증명하라

첫째,
if n=2 and A[0] > A[1]
then swap A[0] and A[1]
위의 구문에 의해, n=2일 때는 A[0]과 A[1]이 소팅이 됨을 알 수 있다.

둘째,
어떤 원소x는 최종 소팅의 결과에서 C구간에 있을 원소라고 하자. 배열 전체에서 x보다 작은 원소의 개수를 a라고 하고, [A B]구간에서 x보다 작은 원소의 개수를 b라고 하자. a-b는 C구간의 원소의 개수보다 클 수 없으므로, a-b <= 1/3n 이다.

원소의 개수 n=2부터 n-1 까지 소팅이 된다고 가정할 때, 소팅 후 C의 위치에 있어야 하는 어떤 원소x가 ①연산 후 A에 위치하는 경우에만 소팅이 재대로 되지 않고, 그러한 x가 없는 경우 항상 소팅이 재대로 된다는 것을 경우를 따져보면 원소 n개에 대한 소팅이 성립함을 알 수 있다.

원소 x가 소팅 후 C의 위치에 있어야 하기 때문에, 배열 전체에서 x보다 작은 원소의 개수 a는, (a) >= 2/3*n 임을 알 수 있다.

문제 3: 다음 정렬 알고리즘이 실제로 정렬에 항상 성공한다는 것을 증명하라

③연산 이후에 C의 위치에 있기 위해서는 ①연산 이후에 B의 위치에 있어야 하는데, ①연산 이후에 B의 위치가 아닌 A의 위치에 있다고 가정한다면, 이 원소 x는 C의 위치로 갈 수 없다고 할 수 있다.

이때 입력 상태의 [A B] 범위에서 x보다 작은 원소의 개수 b는 (b) < 1/3 * n이다. (왜냐하면 정렬 이후 A에 있었기 때문에) 따라서 (a) - (b) > 1/3*n 인데, 앞서 증명한 (a) - (b) <= 1/3 * n 과 모순이 되므로, 그러한 x는 없다는 것을 알 수 있다.

문제 4: 문제3 소팅 알고리즘에서 수행하는 Swap의 횟수는 최대 몇 번인가?

Master Theorem (마스터 정리)

$$T(n) = 1 + 3T\left(\frac{n}{\frac{3}{2}}\right)$$
$$= 1 + 3 + 3^2 \cdot T\left(\frac{n}{(\frac{3}{2})^2}\right)$$
$$= 1 + 3 + 3^2 + \cdots + 3^{\log_{\frac{3}{2}} n}$$
$$= \frac{3^{\log_{\frac{3}{2}} n} - 1}{3-1}$$
$$= \frac{1}{2} \cdot 3^{\log_{\frac{3}{2}} n} - \frac{1}{2}$$

$$O(T(n)) = O\left(n^{\log_{\frac{3}{2}} 3}\right) \approx O\left(n^{2.7}\right)$$

최대 $n^{2.7}$번 Swap할 수 있다.

Master Theorem

$T(n) = aT\left(\frac{n}{b}\right) + f(n)$

$f(n) = 0(n^c)$

$\log_b a > c$

$T(n) = n^{\log_b a}$

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

동적 계획법

약간의 설명

- 간단하게 설명하면 재귀 함수에서 동일한 입력의 함수 호출이 반복적으로 일어날 때 그 결과 값을 저장해 두고 불러 쓰는 것임 (Memoization)
- 최초 입력에서 파생되는 모든 가능한 입력에 대한 답을 모두 저장할 수 있는 메모리가 있어야 함
- 단순히 재귀에서 저장된 값을 찾아보는 것으로도 가능하지만, 결과 값을 순서를 정해서 계산할 수 있음 (Dynamic Programming)
- 동적 계획법 알고리즘을 슈도코드로 작성하고, 정확성 증명 및 시간 복잡도를 계산하는 연습이 필요함

문제 1: Memoization피보나치 수열 : $F(n)=F(n-1)+F(n-2)$, $F(1)=F(2)=1$

- 힌트 : 계산되는 값이 n 가지 밖에 없으므로 이 값들을 저장할 수 있는 배열을 만들어 두고 재귀 호출에 들어가기 전에 값이 있는 지 확인

슈도코드

```
fibo(n)
    if memoization[n] == null
        memoization[n] = fibo(n-1) + fibo(n-2)
    return memoization[n]
```

피보나치 수는 아래와 같은 점화식으로 정의되어 있다.

$$\mathrm{F_n} := \begin{cases} 0 & if\ \mathrm{n} = 0 \\ 1 & if\ \mathrm{n} = 1 \\ \mathrm{F}_{n-1} + F_{n-2} & if\ \ \mathrm{n} > 1 \end{cases}$$

위의 코드에서 n이 0 또는 1일 경우, Base case로 처리가 된다. n의 값이 0 또는 1 이외의 값이 들어올 경우 $\mathrm{Fibonacci(n-1)}, \mathrm{Fibonacci(n-2)}$의 호출을 통해 그 값을 계산하고 이를 저장하는 것을 확인할 수 있다.

Time complexity)

$O(n)$

문제 2: Dynamic Programming 피보나치 수열: $F(n)=F(n-1)+F(n-2)$

- 힌트 : 작은 값부터 순서대로 계산

```
Fibonacci(n)
        F[0]  = 0
        F[1] = 1

        for i : 2 -> n
                F[i] = F[i-1] + F[i-2]
        return F[n]
```

Time complexity)

$O(n)$

문제 2: Dynamic Programming 피보나치 수열: $F(n)=F(n-1)+F(n-2)$

모든 자연수 n에 대하여 $F(n)$이 성립하는 것을 증명하기 위해, 수학적 귀납법을 이용한다.
$n = 1$일 때,
$F(1) = 1$ 이므로 성립한다.
$n = k$일 때,
$F(k)$가 성립한다고 가정하자.
즉 $F(k) = F(k-1) + F(k-2)$ 는 성립한다고 가정하자.
양변에 $F(k-1)$ 을 더하면,

$$F(k) + F(k-1) = F(k-1) + F(k-2) + F(k-1)$$
$$= F(k-1) + F(k)$$
$$= F(k+1)$$

이므로 모든 자연수 n에 대하여 성립함을 알 수 있다.

Time complexity)
$O(n)$

이론

문제 3: n개의 행렬을 곱하려고 한다. 크기가 $a \times b$인 행렬과 크기가 $b \times c$인 행렬을 곱하는 데 드는 계산량은 $a \times b \times c$라고 한다. n개의 행렬들을 곱하는데 필요한 계산량을 최소화 하는 순서를 찾는 알고리즘을 작성하라. 행렬들의 크기는 다르고, 입력으로 주어진다고 가정하라. 물론 곱하기가 가능한 크기들만 주어진다.

n개의 행렬을 곱하는 데 드는 계산량은 행렬의 곱을 구하기 위해 수행된 곱셈 수이다. 이는 행렬 곱셈의 결합 법칙을 성질을 잘 이용해야 한다. 예를 들어, 각각 행렬의 차원이 $(100 \times 1), (1 \times 100), (100 \times 1), (1 \times 100)$로 다른 $\mathrm{M}_1,\ \mathrm{M}_2,\ \mathrm{M}_3, \mathrm{M}_4$ 가 있다고 가정하자.

$$\mathrm{M}_1(\mathrm{M}_2(\mathrm{M}_3\mathrm{M}_4)) = 30{,}000$$
$$\mathrm{M}_1((\mathrm{M}_2\mathrm{M}_3)\mathrm{M}_4) = 10{,}200$$
$$(\mathrm{M}_1(\mathrm{M}_2\mathrm{M}_3))\mathrm{M}_4 = 10{,}200$$
$$((\mathrm{M}_1\mathrm{M}_2)\mathrm{M}_3)\mathrm{M}_4 = 30{,}000$$
$$(\mathrm{M}_1\mathrm{M}_2)(\mathrm{M}_3\mathrm{M}_4) = 102{,}000$$

위와 같이 곱셈의 순서와 방법에 따라서 연산의 횟수가 변화할 수 있다.
즉, 두 개의 행렬이 하나의 새로운 행렬도 대체되면 다시 n-1개의 행렬을 곱하는 문제로 변환된다.
또한 중복되어 계산되는 부분은 Memoization기법을 통해 줄일 수 있다.

따라서 점화식

$$\mathrm{C}(\mathrm{i},\mathrm{j}) = min\ \{\mathrm{C}(\mathrm{i},\mathrm{k}) + \mathrm{C}(\mathrm{k}+1,\mathrm{j}) + \mathrm{D}(\mathrm{i}-1) \times \mathrm{D}(\mathrm{k}) \times \mathrm{D}(\mathrm{j})\}, \qquad \mathrm{i} \le \mathrm{k} \le \mathrm{j}-1\ )$$

로 정의할 수 있다.

실습

논리와 증명

- 문제 12

기초수식

- 문제 4

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

1 다음 중 항진명제는?

a) $p \wedge \neg p$
b) $p \vee \neg p$
c) $p \rightarrow q$
d) $\neg(p \vee q)$

2 명제 $p \rightarrow q$의 대우는?

a) $q \rightarrow p$
b) $\neg p \rightarrow \neg q$
c) $\neg q \rightarrow \neg p$
d) $p \vee q$

3 집합 A={1,2,3},B={3,4,5} 일 때, $A \cap B$는?

a) {1,2,3,4,5}
b) {3}
c) {1,2}
d) { }

4 다음 함수는 어떤 값을 계산하는가?

```
def f(n):
    if n == 0:
        return 1
    return n * f(n-1)
```

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

5 동적 계획법(DP) 기법의 핵심 아이디어는?

a) 무작위 탐색
b) 동일한 부분 문제의 반복 해결 방지
c) 그리디 선택
d) 무한 루프 방지

6 피보나치 수열을 재귀로 정의할 때, 점화식은?

F(0)=0,F(1)=1
F(n) =

7 p : T, q : F 일 때, $p \wedge (p \vee q)$의 진릿값은?

a) T
b) F

8 다음 중 “진릿값”에 대한 설명으로 옳은 것은?

a) 명제가 참일 때만 존재하는 값
b) 명제의 참(True)과 거짓(False)을 나타내는 값
c) 수학적 수식에서 변수의 개수를 의미
d) 집합의 원소 개수를 나타내는 값

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

9 명제 p,q가 있을 때, $p \wedge q$의 진리표에서 참(True)이 되는 경우는?

a) p,q 둘 다 참일 때
b) p,q 둘 다 거짓일 때
c) 둘 중 하나만 참일 때
d) 항상 참이다

10 명제 p가 "오늘은 비가 온다"일 때, $\neg p$의 진릿값이 참(True)이 되는 경우는?

a) 실제로 비가 올 때
b) 실제로 비가 오지 않을 때
c) 항상 참이다
d) 항상 거짓이다

11 명제 p,q가 있을 때, $p \vee q$의 진리표에서 거짓(False)이 되는 경우는?

a) p,q 둘 다 참일 때
b) p,q 둘 다 거짓일 때
c) 둘 중 하나만 참일 때
d) 항상 참이다

12 명제 p,q에 대해 $p \rightarrow q$가 거짓이 되는 경우는?

a) p가 참이고 q가 거짓일 때
b) p가 참이고 q가 참일 때
c) p가 거짓이고 q가 참일 때
d) p가 거짓이고 q가 거짓일 때

확인문제

정답 및 해설

1. b) p∨¬p 2. c) ¬q→¬p 3. b) {3} 4. 팩토리얼 5. b) 동일한 부분 문제의 반복 해결 방지 6. F(n)=F(n−1)+F(n−2) 7. a) T 8. b) 명제의 참(True)과 거짓(False)을 나타내는 값 9. a) p,q 둘 다 참일 때 10. b) 실제로 비가 오지 않을 때 11. b) p,q 둘 다 거짓일 때 12. a) p가 참이고 q가 거짓일 때

1. 항상 참인 명제는 p∨¬p 입니다.

2. 대우는 원래 명제와 항상 같은 진릿값을 갖습니다.

3. 교집합은 두 집합에 공통으로 포함된 원소 집합입니다.

4. 팩토리얼을 계산하는 재귀함수 코드입니다.

5. DP는 메모이제이션을 통해 부분 문제를 재사용합니다.

6. F(n)=F(n−1)+F(n−2),F(0)=0,F(1)=1

7. $(p \vee q)$는 T 이므로, $p \wedge (p \vee q)$도 T입니다.

8. 진릿값은 명제가 참인지 거짓인지를 나타내는 값으로, 참(True)과 거짓(False) 두 가지 뿐입니다.

9. 논리곱은 두 명제가 모두 참일 때만 참입니다.

10. 부정은 원래 명제가 거짓일 때 참이 됩니다.

11. 논리합은 두 명제가 모두 거짓일 때만 거짓입니다.

12. 조건명제는 오직 “참 → 거짓”일 때만 거짓이 됩니다..

활동정리

핵심키워드

개념	설명	예시
Hard logic	형식 논리와 증명을 바탕으로 한 사고 방식	짝수 n의 제곱은 짝수. n=2k (k는 정수)를 대입해 증명
명제	참이나 거짓을 알 수 있는 식이나 문장	1+1 = 3, 서울은 대한민국의 수도
log n	컴퓨터 분야의 로그는 밑이 2	x=log$n$이면 2^x=n
조합론	경우의 수를 따지는 문제들	5명의 중에 2명을 뽑는 경우의 수
Memoization	동일한 입력의 함수 호출이 반복적으로 일어날 때 그 결과 값을 저장	메모이제이션을 이용한 피보나치 수 함수
Dynamic Programming	작은 부분 문제의 결과를 저장해 재사용	DP 테이블을 이용한 피보나치 수 계산

<표16_SW문제해결 응용_컴퓨팅사고력_핵심키워드>

활동정리

오늘 공부한 내용 요약 및 정리

- **논리(Hard Logic)란?**
  - 엄밀한 형식 논리와 증명을 바탕으로 한 사고 방식
  - 수식과 증명이 사용됨

- **명제**
  - 참이나 거짓을 알 수 있는 식이나 문장

- **조건명제**
  - p, q가 명제일 때, 명제 p가 조건(또는 원인), q가 결론(또는 결과)로 제시되는 명제
  - 조건명제의 대우는 조건명제와 진릿값이 같음

활동정리

오늘 공부한 내용 요약 및 정리

- **증명**
  - 어떤 명제가 참임을 논리적으로 확인하는 과정

- **조합론**
  - 경우의 수를 따지는 문제들

- **재귀**
  - 자기 자신의 입력과 동일한 입력으로 자기 자신을 호출하면 끝나지 않지만, 다른 입력으로 호출하면 끝날 수 있음

- **동적 계획법**
  - 자기 자신의 입력과 동일한 입력으로 자기 자신을 호출하면 끝나지 않지만, 다른 입력으로 호출하면 끝날 수 있음

활동정리

'Problem Solving 문제를 봤는데 어떻게 풀어야 하는지 전혀 감이 오지 않는다면?

1. [ ] 정확하게 확인하는 과정에 대한 연습이 필요합니다
2. 정확하게 확인하는 과정을 수많은 세월 동안 정리해 둔 것이 [ ] 기법입니다

활동정리

'Problem Solving 문제들 봤는데 어떻게 풀어야 하는지 전혀 감이 오지 않는다면?

1. 논리적으로 정확하게 확인하는 과정에 대한 연습이 필요합니다
2. 정확하게 확인하는 과정을 수많은 세월 동안 정리해 둔 것이 “증명” 기법입니다

버블 소트가 항상 정렬할 수 있음을 귀납적으로 증명
n이 짝수이면 n^2도 짝수임을 직접 증명

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

감사합니다